

MANUAL TECNICO

Estructura General del Programa:

El sistema fue hecho en visual estudio code con el lenguaje de C++ utilizando programación orientada a objetos(POO). El proyecto está desarrollado de una forma modular, separando declaraciones de clases (.h) e implementaciones (.Cpp).

Diagrama de Componentes UML:

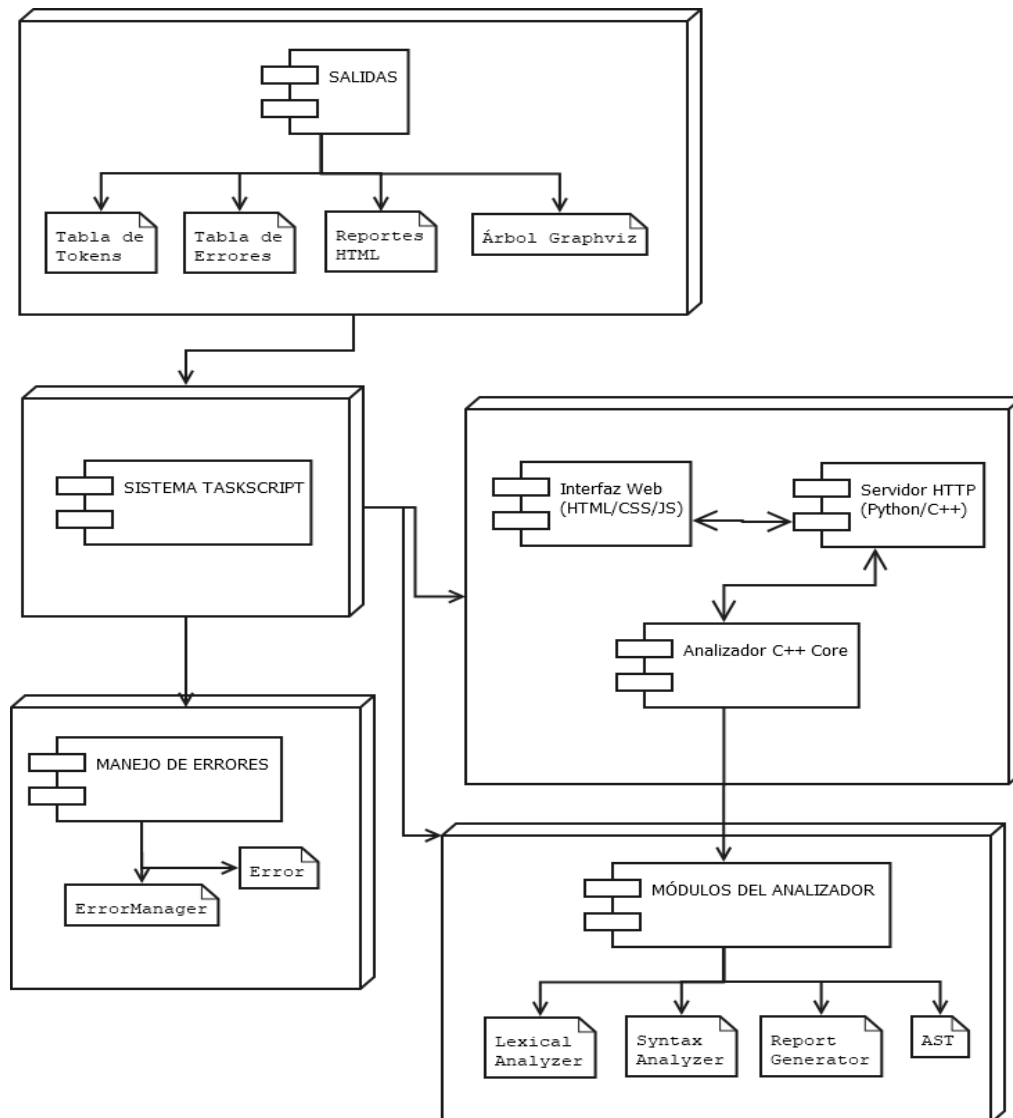


Diagrama de Clases Completo:

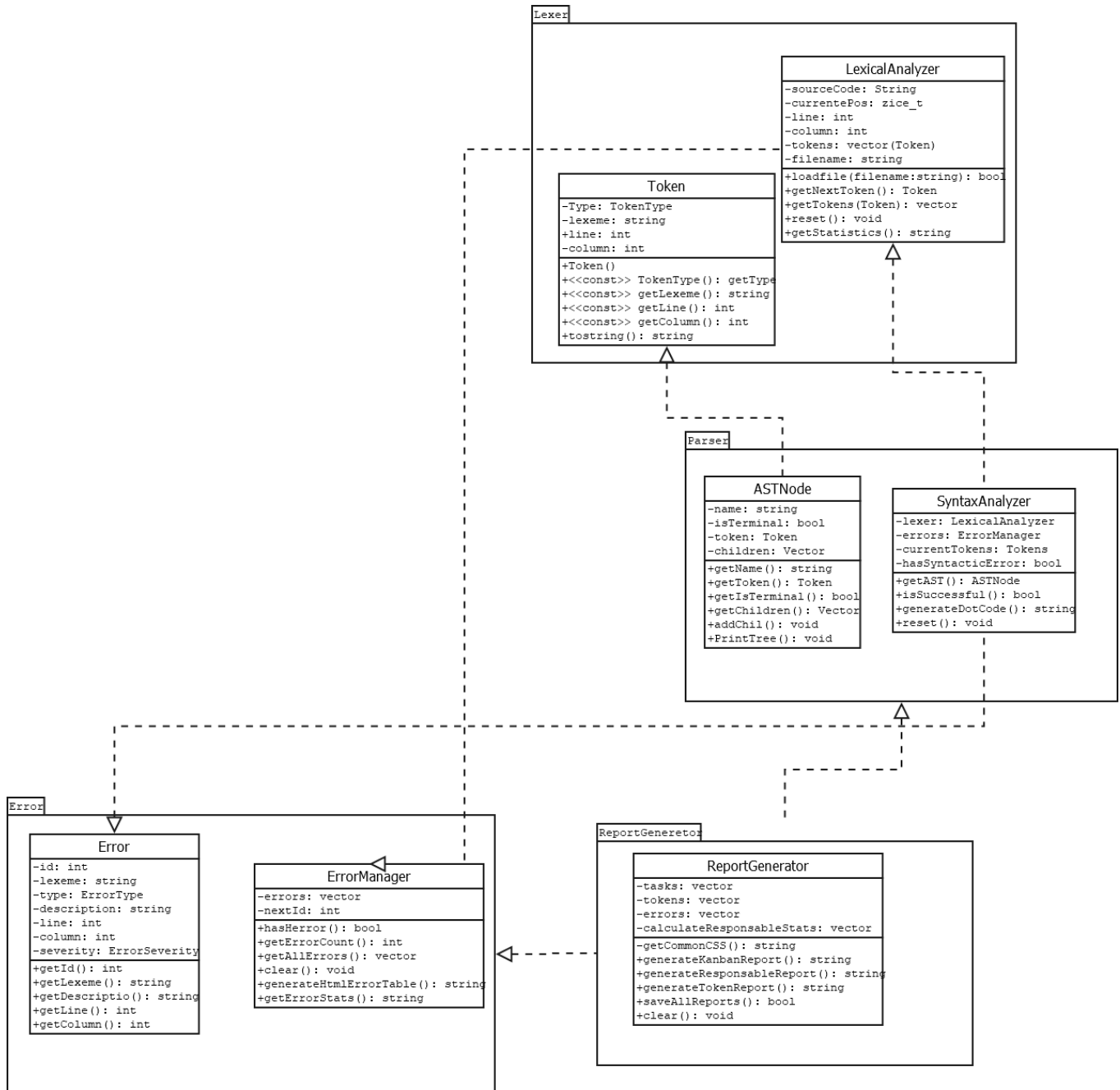


Diagrama del AFD con todos los estados y transiciones:

Básicas:

L = [a-zA-Z]
N = [0-9]
S = [{ } ; , \[\] : ""]
W = [\t \n \r]

Palabras Reservadas:

TABLERO
COLUMNA
TAREA
PRIORIDAD
RESPONSABLE
FECHA_LIMITE

Enum de prioridad:

ALTA
MEDIA
BAJA

Literales:

IDENTIFICADOR
CADENA
FECHA
ENTERO

Delimitadores y operadores:

LLAVE_ABRE
LLAVE_CIERRA
CORCHETE_ABRE,
CORCHETE_CIERRA,
DOS_PUNTOS,
COMA,
PUNTO_Y_COMA,

Token especial para indicar fin del archivo:

FIN_DE_ARCHIVO

Token para caracteres no reconocidos:

DESCONOCIDO

Generalización:

- **ER:** $((((L)^*|N^*|)(L)^*|S^*))\#$
- Diagrama:

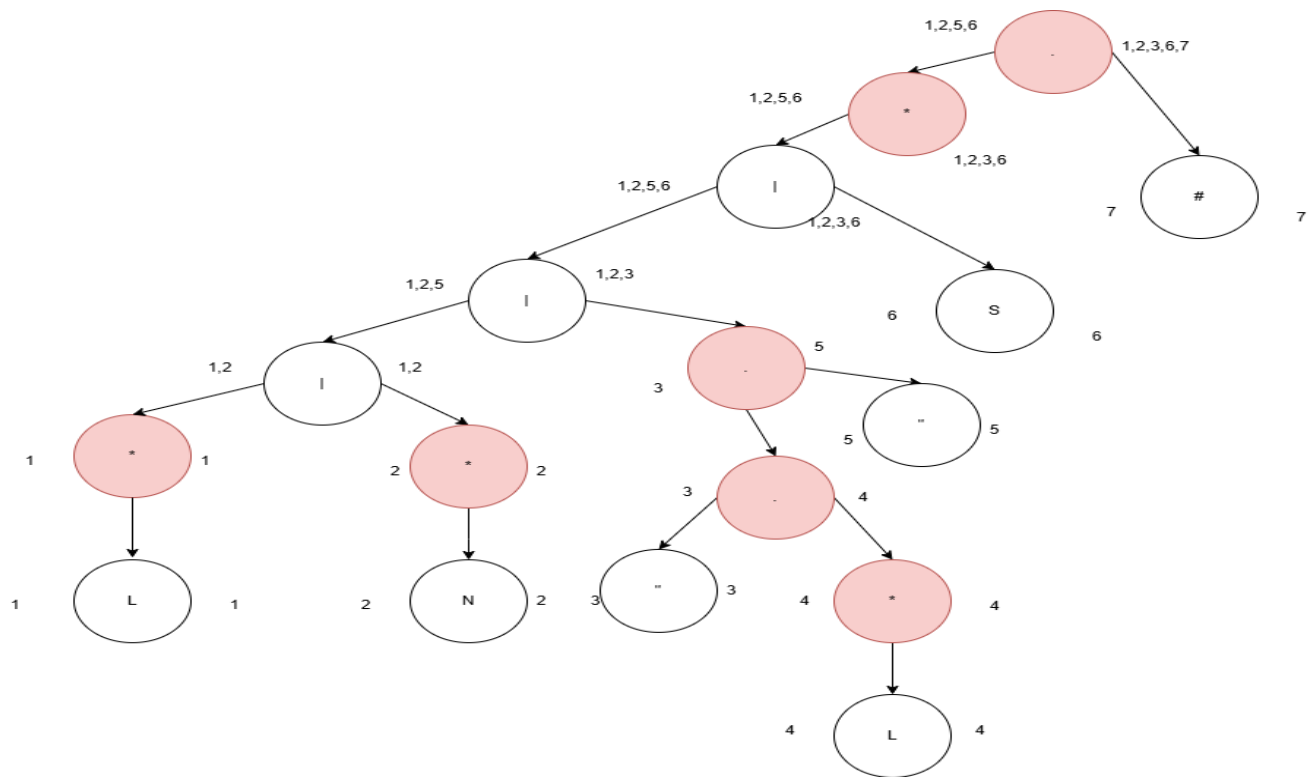
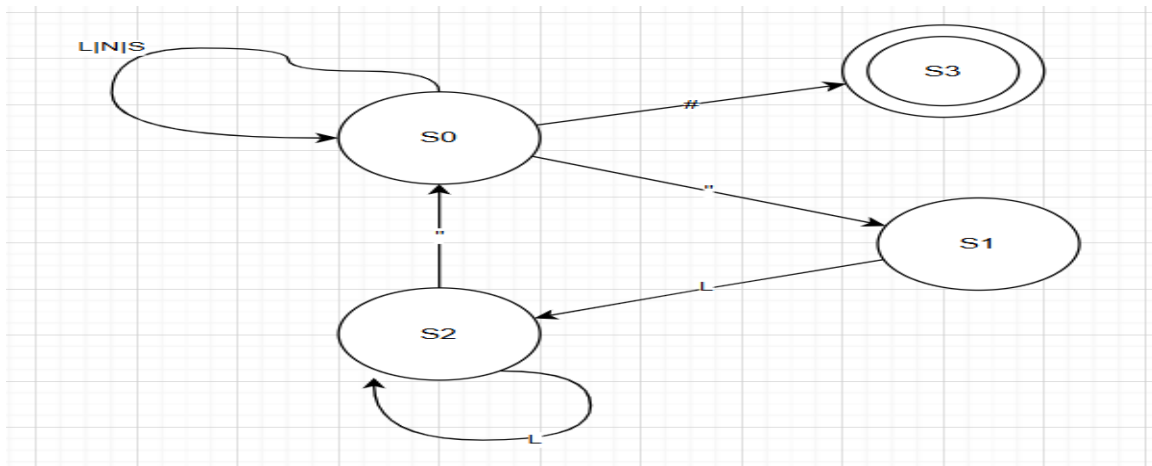


Tabla:

HOJA	TERMINAL	SIGUIENTES
1	L	1,2,3,6
2	N	1,2,3,6
3	"	4
4	L	4,5
5	"	1,2,3,6
6	S	1,2,3,6
7	#	-

Transiciones del autómata:

$S0 = \{1,2,3,6\}$	$L = S0 = \{1,2,3,6\}$
$S0 = \{1,2,3,6\}$	$N = S0 = \{1,2,3,6\}$
$S0 = \{1,2,3,6\}$	$" = S0 = \{1,2,3,6\}$
$S0 = \{1,2,3,6\}$	$S = S0 = \{1,2,3,6\}$
$S1 = \{4\}$	$L = S2\{4,5\}$
$S2\{4,5\}$	$" = S0 = \{1,2,3,6\}$
$S3 = 7$	-



Fecha (Formato: AAAA-MM-DD):

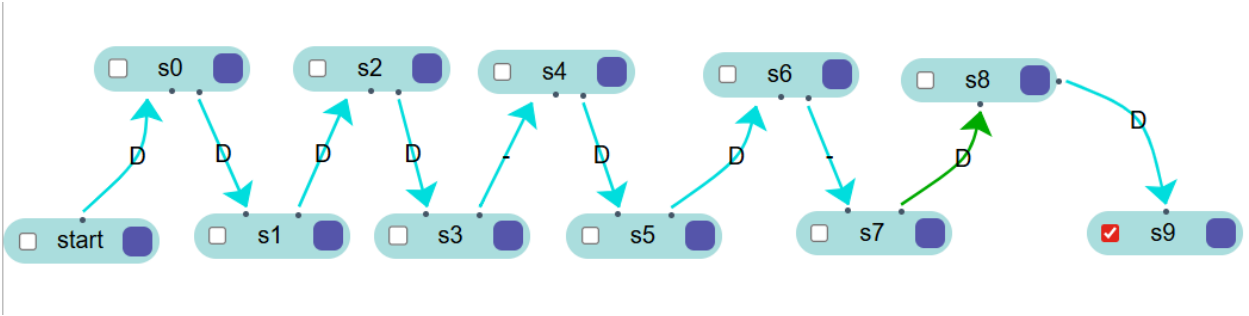
ER:

$([0-9][0-9][0-9][0-9]) - ([0-9][0-9]) - ([0-9][0-9])$

D: Dígitos, X= Trampa

$\Sigma = \{D, -\}$

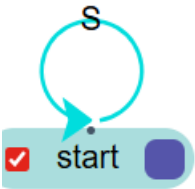
	D	-
->Star	S0	X
S0	S1	X
S1	S2	X
S1	S3	X
S3	X	S4
S4	S5	X
S5	S6	X
S6	X	S7
S7	S8	X
S8	S9	X
*S9	X	X



Símbolos

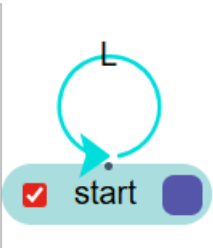
ER: LLAVE_ABRE = "{"
LLAVE_CIERRA = "}"
CORCHETE_ABRE = "["
CORCHETE_CIERRA = "]"
DOS_PUNTOS = ":"
COMA = ","
PUNTO_COMA = ";"

q0	S--> q0
----	---------



PRIORIDAD
ER: PRIORIDAD = "ALTA|MEDIA|BAJA"

q0	L--> q0
----	---------



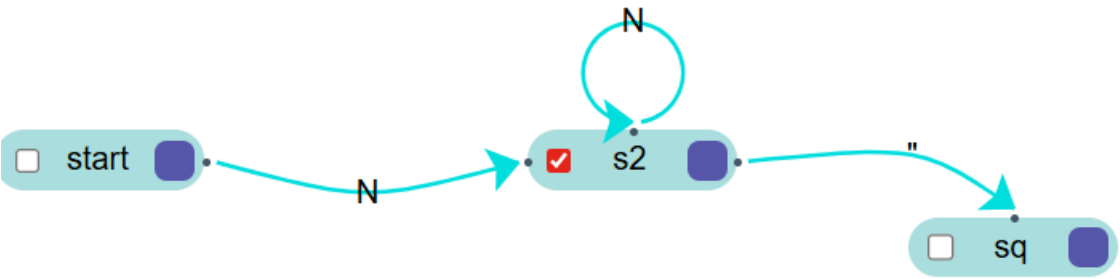
ENTERO

ER:

N+

qE = estado de error

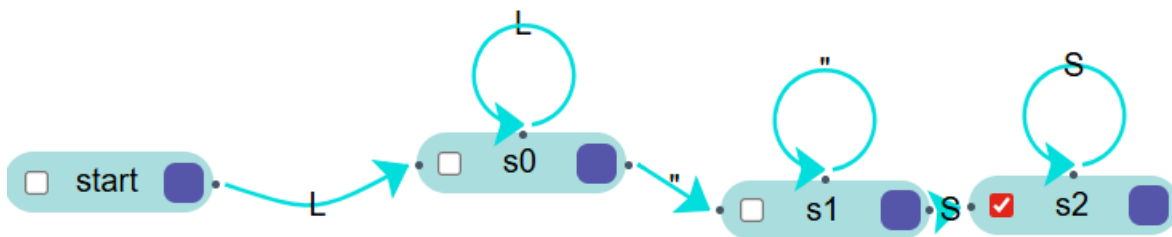
q0	N--> q2
q2	N--> q2
q2	"---> qE
q2	"---> qE



TABLERO

ER: $L^+ | " " | L | S^+ |$

q0	$L \rightarrow S0$
S0	$L \rightarrow S0$
S0	$" \rightarrow S1$
S1	$" \rightarrow S1$
S1	$S \rightarrow S2$
S2	$S \rightarrow S2^*$



Gramática libre de contexto completa:

Notación

- No terminales: $\langle \rangle$
- Terminales: MAYÚSCULAS (tokens)
- Producción: $::=$
- Alternativas: $|$

<programa> ::= TABLERO CADENA LLAVE_ABRE <columnas> LLAVE_CIERRA
PUNTO_Y_COMA

<columnas> ::= <columna> <columnas>
| <columna>

<columna> ::= COLUMNA CADENA LLAVE_ABRE <tareas> LLAVE_CIERRA
PUNTO_Y_COMA

<tareas> ::= <tarea> COMA <tareas>
| <tarea>

<tarea> ::= TAREA DOS_PUNTOS CADENA CORCHETE_ABRE <atributos>
CORCHETE_CIERRA

<atributos> ::= <atributo> COMA <atributos>
| <atributo>

<atributo> ::= PRIORIDAD DOS_PUNTOS <prioridad>
| RESPONSABLE DOS_PUNTOS CADENA
| FECHA_LIMITE DOS_PUNTOS FECHA

<prioridad> ::= ALTA
| MEDIA
| BAJA

Gramática con Eliminación de Recursión Izquierda:

$\langle \text{programa} \rangle ::= \text{TABLERO CADENA LLAVE_ABRE } \langle \text{columnas} \rangle \text{ LLAVE_CIERRA}$
 PUNTO_Y_COMA

$\langle \text{columnas} \rangle ::= \langle \text{columna} \rangle \langle \text{columnas_aux} \rangle$

$\langle \text{columnas_aux} \rangle ::= \langle \text{columna} \rangle \langle \text{columnas_aux} \rangle$
 $\mid \epsilon$

$\langle \text{columna} \rangle ::= \text{COLUMNA CADENA LLAVE_ABRE } \langle \text{tareas} \rangle \text{ LLAVE_CIERRA}$
 PUNTO_Y_COMA

$\langle \text{tareas} \rangle ::= \langle \text{tarea} \rangle \langle \text{tareas_aux} \rangle$

$\langle \text{tareas_aux} \rangle ::= \text{COMA } \langle \text{tarea} \rangle \langle \text{tareas_aux} \rangle$
 $\mid \epsilon$

$\langle \text{tarea} \rangle ::= \text{TAREA DOS_PUNTOS CADENA CORCHETE_ABRE } \langle \text{atributos} \rangle$
 CORCHETE_CIERRA

$\langle \text{atributos} \rangle ::= \langle \text{atributo} \rangle \langle \text{atributos_aux} \rangle$

$\langle \text{atributos_aux} \rangle ::= \text{COMA } \langle \text{atributo} \rangle \langle \text{atributos_aux} \rangle$
 $\mid \epsilon$

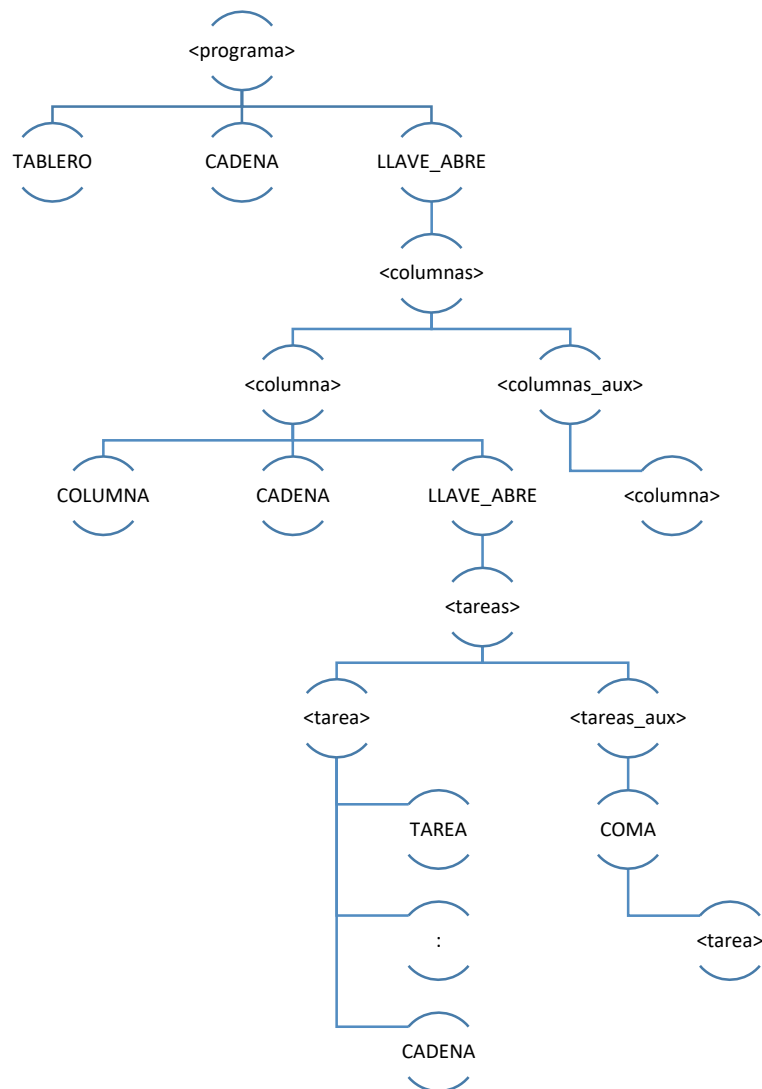
$\langle \text{atributo} \rangle ::= \text{PRIORIDAD DOS_PUNTOS } \langle \text{prioridad} \rangle$
 $\mid \text{RESPONSABLE DOS_PUNTOS CADENA}$
 $\mid \text{FECHA_LIMITE DOS_PUNTOS FECHA}$

$\langle \text{prioridad} \rangle ::= \text{ALTA}$
 $\mid \text{MEDIA}$
 $\mid \text{BAJA}$

```

programa      = "TABLERO", CADENA, "{", columnas, "}", ";" ;
columnas      = { columna } ;
columna       = "COLUMNA", CADENA, "{", tareas, "}", ";" ;
tareas        = tarea, { ",", tarea } ;
tarea         = "tarea", ":", CADENA, "[", atributos, "]" ;
atributos     = atributo, { ",", atributo } ;
atributo      = ( "prioridad", ":", prioridad )
               | ( "responsable", ":", CADENA )
               | ( "fecha_limite", ":", FECHA ) ;
prioridad     = "ALTA" | "MEDIA" | "BAJA" ;

```



Descripción del Parser

El analizador sintáctico es un Parser Descendente Recursivo (Recursive Descent Parser). Este tipo de parser pertenece a la familia de los analizadores sintácticos top-down, donde el proceso de análisis comienza desde el símbolo inicial de la gramática y procede hacia los símbolos terminales.

Características Principales:

Característica	Descripción
Enfoque	Top-down (descendente)
Técnica	Recursivo con retroceso (backtracking)
Implementación	Una función por cada no-terminal de la gramática
Manejo de errores	Recuperación mediante sincronización
Complejidad	$O(n)$ en tiempo lineal

Correspondencia Gramática-Parser:

Producción Gramatical	Función en C++
<programa>	parsePrograma()
<columnas>	parseColumnas()
<columna>	parseColumna()
<tareas>	parseTareas()
<tarea>	parseTarea()
<atributos>	parseAtributos()
<atributo>	parseAtributo()
<prioridad>	parsePrioridad()

Estrategia de Recuperación de Errores:

Se implementó la estrategia de sincronización (panic mode), considerada la más adecuada para lenguajes de programación y procesadores de lenguaje.

Principios de la Estrategia:

- No detener el análisis ante el primer error
- Reportar todos los errores encontrados
- Sincronizar en puntos seguros de la gramática
- Continuar el análisis desde un estado válido

Error Detectado	Acción de Recuperación	Sincronización en
Falta TABLERO	Saltar hasta TABLERO	TABLERO
Falta : después de tarea	Saltar hasta : o '['	DOS_PUNTOS, CORCHETE_ABRE
Falta '[' en atributos	Saltar hasta '['	CORCHETE_ABRE
Falta ']' en atributos	Saltar hasta ']' o '}'	CORCHETE_CIERRA, LLAVE_CIERRA
Falta ; en columna	Saltar hasta ';' o '}'	PUNTO_Y_COMA, LLAVE_CIERRA
Atributo inválido	Saltar hasta próxima coma o ']'	COMA, CORCHETE_CIERRA

Justificación de Decisiones de Diseño:

¿Por qué Parser Descendente Recursivo?

Razón	Justificación
Simple	La gramática de TaskScript es LL(1), ideal para parsers recursivos
Mantenible	Cada regla gramatical tiene su función, fácil de modificar
Depuración	Las funciones son independientes y trazables
Restricción	Prohibido usar herramientas de generación de parsers (ANTLR, Bison)

¿Por qué la gramática es LL(1)?

La gramática diseñada cumple con las propiedades LL(1):

- No tiene recursión izquierda (fue eliminada)
- Está factorizada por la izquierda
- Los conjuntos PRIMEROS y SIGUIENTES son disjuntos

Recuperación de Errores (vs Detención):

Criterio	Decisión Tomada	Alternativa Rechazada
Comportamiento	Continuar tras error	Detener en primer error
Reporte	Acumular todos los errores	Reportar solo el primero
Complejidad	Mayor (requiere sincronización)	Menor

Justificación: El proyecto especifica que "el sistema no debe detenerse ante el primer error".

Uso de WEB:

El proyecto del analizador TaskScript fue desarrollado utilizando una arquitectura basada en tecnologías web (HTML, CSS, JavaScript) con un servidor backend en Python, en lugar de la interfaz gráfica Qt sugerida originalmente. Esta decisión se justifica por razones de eficiencia de desarrollo, portabilidad, facilidad de depuración, mantenibilidad y cumplimiento de requisitos.

La decisión de utilizar tecnologías web (HTML, CSS, JavaScript) con un servidor Python en lugar de Qt se justifica por:

- Cumplimiento de requisitos: Los reportes HTML ya son parte del proyecto
- Portabilidad: Funciona en cualquier sistema operativo sin recompilar
- Eficiencia de desarrollo: Tiempo de compilación cero, depuración sencilla
- Madurez de herramientas: HTML/CSS/JS tienen mejor soporte para interfaces modernas
- Separación de responsabilidades: El analizador C++ se mantiene puro y portable
- Visualización de resultados: Los reportes HTML son nativos y compatibles con el proyecto